PROJECT OVERVIEW

Main Yuk

One device, 12 party games, 245 cards for hanging out together.

Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihan-evanza.vercel.app

ROLE

Personal project · game engine, frontend, mobile, API, and Rupa components

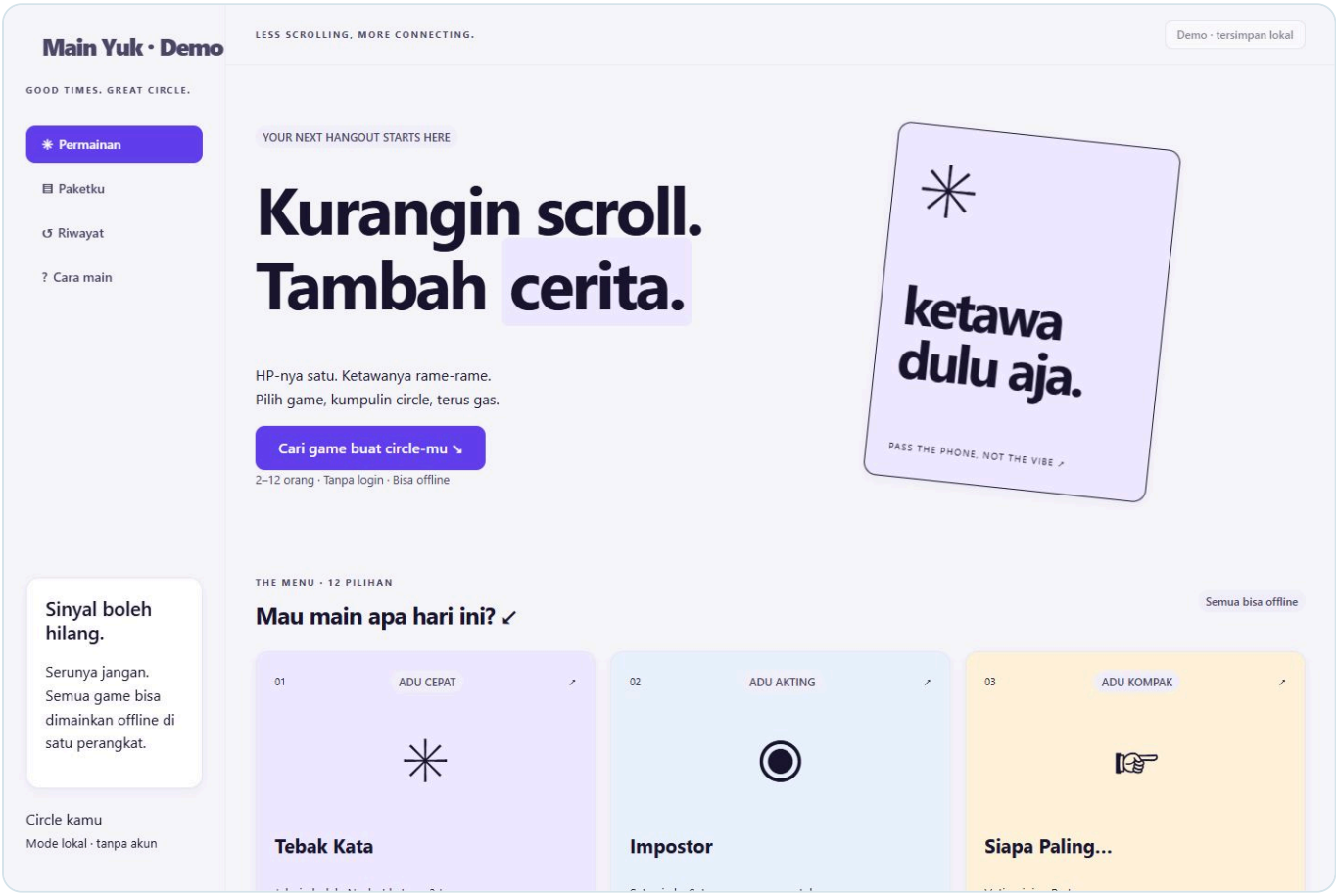
PLATFORMS

Responsive web and React Native; browser-based portfolio demo

LINKS

Live demo: raihan-evanza-main-yuk.vercel.app

React · React Native · TypeScript · Rupa · WatermelonDB · RxJS · Ruby on Rails · PostgreSQL · Vite · Vitest · Playwright



OVERVIEW

Main Yuk turns one phone into a shared game device. Pick a game, enter player names, and follow the turns on screen. Each player does not need an app installation or an account.

A shared TypeScript game engine powers web and mobile. Rupa supplies every screen and form through props and callbacks. WatermelonDB keeps packs and session snapshots locally; the full app can synchronize them through Rails and PostgreSQL.

The portfolio demo uses the same app and components in a separate build without an API or sign-in. Twelve games and 245 cards are available immediately; custom packs and sessions survive reloads in the same browser.

KEY FEATURES

12 games

Word guessing, Impostor, Most Likely To, Charades, Five-Second Answers, Truth or Dare, Would You Rather, Story Chain, Quick Quiz, Forbidden Words, Sequence Memory, and Word Chain.

Custom packs

245 built-in cards, a pack editor, favorites, and format validation for quizzes and forbidden-word cards.

Private handoffs

Secret roles and private choices appear only after the intended player confirms they are ready.

Timers and scoring

Deadlines live in session snapshots, so reloads do not restart a turn timer. The engine validates actions and updates scores.

Resume a session

History stores progress and results. An active session can be reopened after reloading the page.

Shared UI in Rupa

PartyApp, PartyBoard, and PartyNotice are available for web and React Native. The host owns data and rules and passes props to the UI.

ARCHITECTURE & DATA FLOW

Rupa UI
Screens, forms, dialogs, and inputs

→

TypeScript
Controller, presenter, and game transitions

→

WatermelonDB
Local packs, cards, and session snapshots

→

Rails + PostgreSQL
Account sync; full app only

- The portfolio demo uses browser storage and sends no API requests.
- Web and mobile share a game engine and matching prop contracts.
- Rupa is bundled in the demo snapshot, so deploying the portfolio needs neither the Rupa nor Main Yuk repository.

STACK BY LAYER

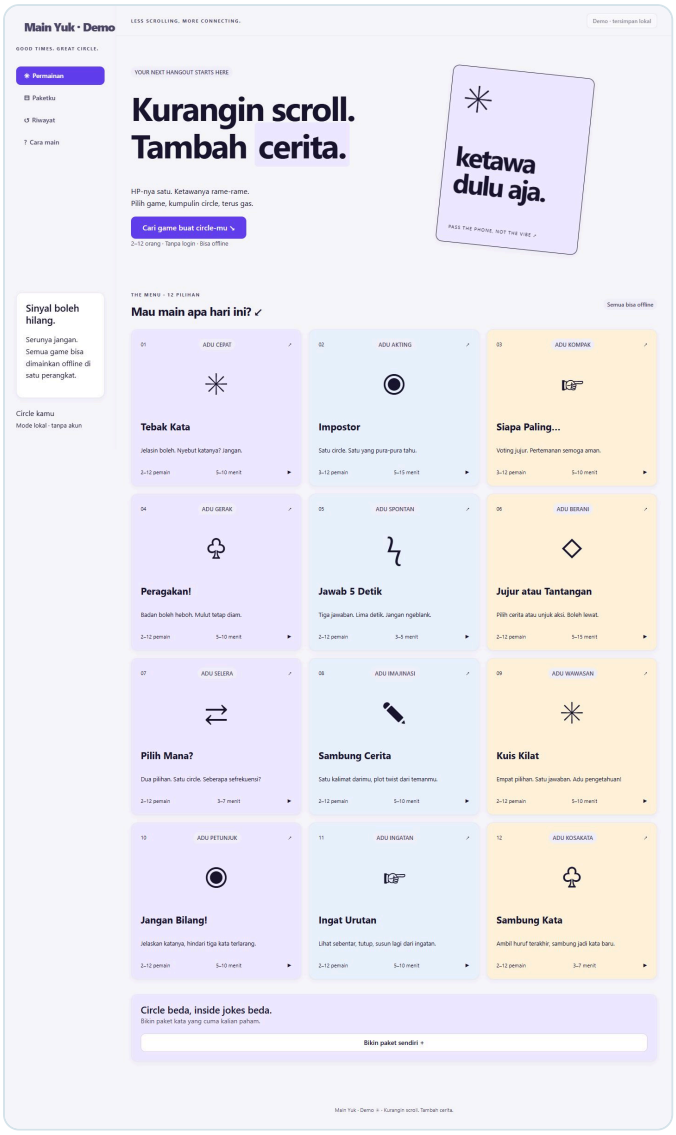
Interface	React · React Native · Rupa · TypeScript
Local data	WatermelonDB · IndexedDB / SQLite · RxJS
Full backend	Ruby on Rails · PostgreSQL · Docker
Testing	Vitest · Playwright · Rails tests · Rupa Storybook

TECHNICAL DECISIONS

1. **Game engine independent of UI**
   Snapshot transitions and presenters are separate from rendering, allowing rules to be tested once and used on both platforms.
2. **Rupa receives props**
   Rupa never accesses the database or API. Text inputs, game boards, and local mode are controlled through data contracts and callbacks.
3. **Separate static demo**
   The demo build disables accounts and service workers, uses its own data namespace, and ships inside the portfolio so deployment needs no backend.

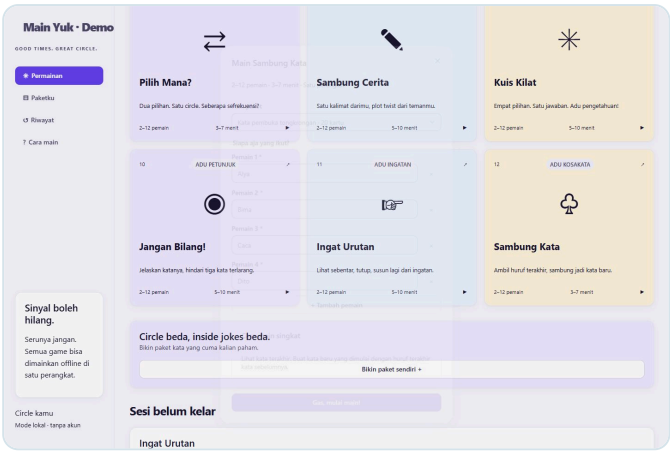
HOW TO USE IT

These steps follow the same order as the tutorial video.



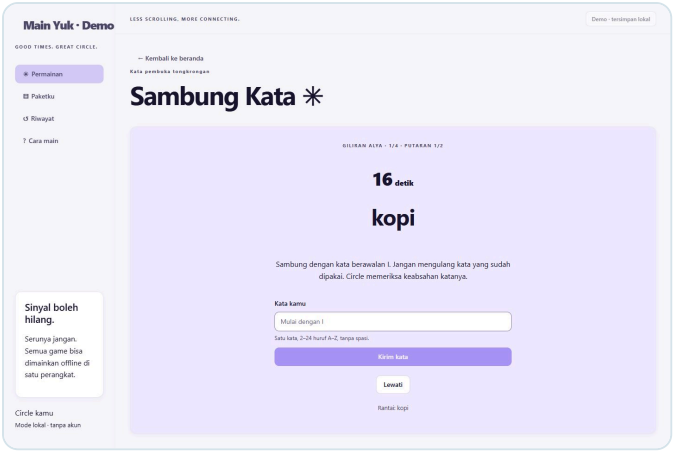
1 Open the demo

Select Demo on the Main Yuk project. The catalog opens immediately without sign-in.



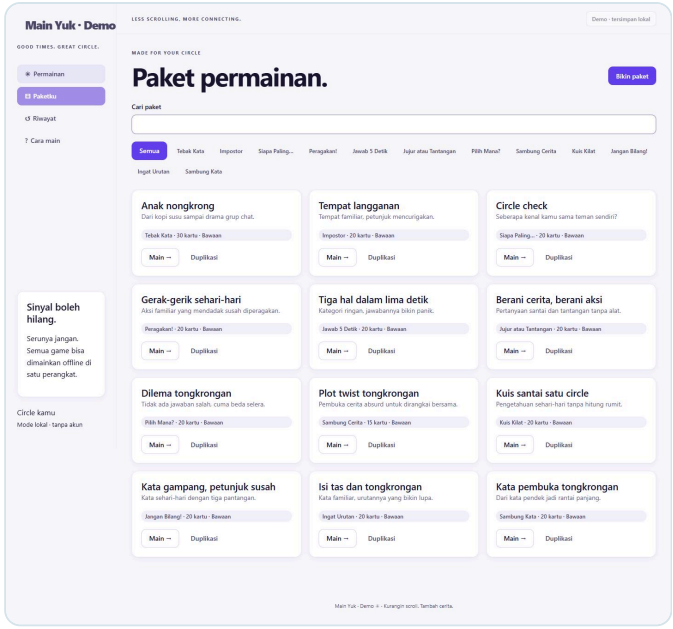
2 Choose a game and players

Open a game, select a pack, and change player names. Player limits follow that game's rules.



3 Follow the instructions

Select Gas, mulai main! and pass the phone as instructed. For Word Chain, type a word starting with the last letter of the previous word.



4 Try a custom pack

Open Paketku and Bikin paket. Enter a title, game type, and at least five cards following the format hints.

5 Resume after reloading

Return home or open Riwayat, then choose a session to continue. Use the same browser and profile.

LIMITATIONS & NOTES

- Games share one device and do not provide network multiplayer.
- The demo uses local browser storage, without sign-in or account sync. Clearing site storage removes its data.
- The demo requires localhost or HTTPS, IndexedDB, and Web Locks. Reloading needs a connection because no service worker is installed.
- Word Chain validates formatting, starting letters, and repeated words; the group judges whether a word exists in the dictionary.
- The demo UI is in Indonesian. Project documentation is available in Indonesian and English.